Plexus Discovery

Progress note — written to be read, not decoded

31 July 2026

Plain English throughout. Organised by phase, because phases are where we stop and you decide.

---

1. What we are doing — two things at once

Track A — build the method. An automated loop whose unit of discovery is a *mechanism*: it proposes a change to the model's **structure** — remove the pushing force, add oriented cell division, wire the growth gate to the reaction instead of the seed — writes down what it expects *before* running, runs the simulation, measures, and records whether it was right.

Parameters are in scope; they are just not discoveries. "What does this mechanism do as you turn it up" is the second question Track A asks of every mechanism, and a sweep is a legitimate experiment. What must never happen is a retune being reported as a new idea — and that is now enforced by arithmetic rather than by prohibition: `comp_hash` excludes parameters, so a composition at a different operating point has the *same identity* and is filed as a point on a known mechanism. The rule holds because the identity function says so, not because an agent was asked to be careful.

And it must be allowed to be curious. A loop that may only test predictions will only ever propose things it can predict, which is how a campaign came to spend nineteen of thirty-four predictions on "nothing will happen" and count eighteen confirmations that a sphere stayed a sphere. An exploratory slot states what it is *varying* and what it will *report*, rather than what it expects. Turning χ up and describing what appears is an experiment; a labyrinth nobody forecast is a finding, not a failed prediction.

Track B — reproduce Okuda's result, as the proof that the method is worth having. Okuda et al. (2018) grew a hollow ball of cells that spontaneously forms tubes, undulations and branches, driven by a chemical pattern on its own surface. We are rebuilding that here, and *the figure is not a side quest — it is the evidence.*

What we may honestly claim, and what we may not. The composition carrying Okuda's coupling is **supplied** to the search, as a starting point on the frontier — it is written down in `reference_recipes()` and the loop begins from it. So the claim is *not* "the loop found the mechanism by itself". What the loop does, and what would be worth publishing, is the **lever map around that mechanism**: which operators are necessary, which are inert, what happens at the extremes, and which of the four morphologies each choice reaches — every one of those a prediction made before the run and scored after it. If a composition the loop reaches by its own edits ever beats the supplied one, that is a stronger claim and it will be stated separately. Until then the honest sentence is: *the mechanism was given; the map around it was earned.*

The two tracks feed each other: the reproduction is the honest test of the method, and the method is what makes the reproduction more than hand-tuning.

2. What we believe about the biology

Nothing *measured*. But we now write down what we assume.

The register of *results* is still empty, and should stay empty until a batch has been taken with instruments that work. Every number we have was measured with at least one broken ruler in the path.

What changed on 31 July is a different thing: the register of **assumptions** is no longer empty. Eleven basics about cell tissue are written down in `discovery_okuda/PREMISES.md` — not a literature review, but the minimum a person needs in order to look at one of our runs and say "that cannot be right". Each is phrased so that a computer can check it against a run, and each one now does (Phase 1).

The distinction matters. A result is something we found out. A premise is something we are *assuming*, stated plainly enough that it can be caught being wrong. Writing the second kind down is not a claim about biology — it is a way of making our own mistakes visible.

3. Two questions instead of one number

Success is not a threshold a single run clears. It is two questions, and they are asked separately because they fail separately.

1. What can we reproduce?	Each of the paper's morphologies is a row, scored qualitatively: *not attempted / attempted and failed / partial / reproduced.* **Today 0 of 4.** Budding is tracked too — our substrate produces it and no criterion ever mentioned it.
2. What do we know?	Already measured, and previously subordinate to the tube gate: map coverage **27%** (3 of 11 cells can state a verdict) — and all of it is routing; solo operator effects are **0 of 8**. Nine claims resolved, ten open, surprise rate **0.375**.

4. The team of agents

`ROLES.md` **is the source of truth, and the code is checked against it in both directions** by `roles.py --check` — the same discipline `PREMISES.md` has for the biology. The roster had reached sixteen roles nobody could account for, because it lived in four places at once: the code, a table in this note, a figure drawn by hand, and everybody's memory. Four descriptions of one thing are three descriptions that are wrong. There is now one, and this note does not repeat it.

The roster was judged by a ledger, not by argument. A live round spent 23.5 minutes across sixteen calls, of which the Proposer took 14.8 — 63% of all agent time — while the Judge and the Referee were *never called at all.* That is not a claim that they were redundant; it is a record showing they had nothing to do. The cause was a borrowed design: *Co-Scientist* runs a tournament of hypotheses because it performs no experiments, so its proposals can only be debated. We measure. Where a tournament would rank two ideas by argument, we run both.

Three acts, and the division that matters. Act 1 proposes, Act 2 measures, Act 3 decides. Within that, the load-bearing split is between what is *arithmetic* and what is *judgement*: the Critic decides whether a batch **can** be run and its refusals are enumerable and arguable-with-never; peer-review decides whether it is **worth** running, which is a judgement, and it advises

rather than vetoes. A judgement that can silently cost twelve GPU-hours is a veto wearing an adviser's name.

A round that produces no evidence is aborted and routes back to Act 1 carrying the refusals. It is recorded in full but does not increment the round counter and enters no coverage denominator — counting it as a normal round is exactly how a log came to tell the Proposer `coverage 0%`, from which it correctly concluded its own ledger was broken. Two consecutive aborts stop the campaign and ask, because a second abort means the refusal reasons were not actionable, which is a fact about us rather than about the search.

As of Phase 8 the loop can be run with no agent in it at all (§8.1), and every hand-off between these roles is declared in `WIRING.md` and checked (§8.2). Those two are what make the paragraphs above testable rather than remembered.

5. The phases

Eight stages. **I stop at the end of each one and wait for you.** That is the partition.

Phase 0 — Pre-flight: fix the instruments — done (28 of 28)

Goal. Make every measurement trustworthy. Nothing runs unattended until this lands. Not one of the twenty-eight items is science; each one makes a question *askable*.

The last two, closed. *Comparing a batch at a common frame* — the per-run evidence horizon was right and did not make a BATCH comparable. If one run tears at frame 300 and another holds to 700, comparing their "final" states puts a torn mesh beside a healthy one and reports the difference as biology: both operands valid, the comparison wrong, which is the shape of every error that has cost this campaign a batch. A batch is now read at one frame, the earliest horizon in it, and the tool reports what that costs — how much of each run is discarded, and a warning when one run's early death is being imposed on everyone. *The explicit vocabulary* — `elongation`, `elongation_peak`, `elongation_at_end`, `elongation_unforced`. Writers emit both names, readers resolve either, the archive is never rewritten: cutting 92 references over in one step leaves a window where a key is written by half the code and read by the other half, and since an absent metric reads as "not measured", the bug would surface as a round that quietly learns nothing.

The repairs worth knowing about

what	what was found
survival test	It relaxed a fresh sphere and returned 1.014 for every run. Now real: **1.415 → 1.325, so 94% of the elongation survived the drivers going off** — the first honest survival number the campaign has produced.
the camera	Growth was drawn as *shrinkage*: apparent size fell to 0.52× on a run that grew by half. A second viewpoint justified itself immediately — in one run **the tube is invisible from the side and obvious from the top**.
the tube bar	**No capsule of any aspect ratio can reach the old bar of 2.0** (ceiling 1.90). The criterion was not asking for a tube; it was asking for the spike artefact.
damage	The blended damage score was counting *cell division*: sliver correlates with tip count at +0.94, while genuinely broken cells stayed at **0 for 300 frames**.
the 1778 ceiling	Arithmetic, not physics: (3552+4)/2 = 1778, and all 32 runs of the overnight study stopped there.
the missing arrow	The emitter never passed the implementation through — the ablation would have reported "no effect" *without ever running the new code.*
the curve plot	Drawn every run since the beginning, referenced **nowhere in the codebase**.
timing	The round reported `calls: 0` for a round that made about 25.

Phase 1 — Teach the loop what a tissue is: the premise gate — done

Why this became its own phase. Ten defects were found in one day, every one by Cedric looking at a picture, and all of the same kind: an operator or an instrument not doing what its name says. The loop checked that runs finished, that the mesh was valid, that the numbers were finite. It never once checked that the thing it had simulated was a tissue.

Eleven basics, each one something a computer can run and fail, in three tiers by cost: read the recipe (instant, refuses a hopeless run before a GPU is touched); read the run (free, goes loudly into the record); and run a small experiment of its own — switch everything off and let the ball sit for forty frames; it must not change size.

A broken premise must be declared, not silenced. Switching off growth to ask what a protrusion is made of genuinely breaks premise 1, and that is science. So a recipe may waive a premise *in writing, with a reason*; an undeclared violation is a hard stop. That is the only thing separating "we tested an idea" from "the settings were wrong".

Every check was proved to fail before being trusted. The one to look at: run against the code as it stood that morning, a resting ball collapses **5.00 → 0.69** in forty frames. That single check, which costs about a minute, would have caught the worst defect of the campaign on day one.

Phase 2 — Make Okuda's settings reachable — done (6 of 6)

The box used to describe the work without listing it, so the count had no visible denominator. All six, with what each one turned out to be:

. item	state	what it turned out to be
0. The ball could not get big enough	done	*Not on the list.* A spring held the shell at the size it was first built at while the cells inside grew sixteen-fold, so the sheet folded through itself. The spring's target now follows what the cells are asking for: the same run grows smoothly to 5 240 cells and stays clean.
1. Widen the out-of-range settings	done	Done, and the new bounds are *derived* rather than chosen: growth sharpness to 49.5 (Okuda needs 10), inhibitor spread to 12.5 (he needs 10), reaction speed unbounded.
2. Replace the mislabelled length-scale	done	The quantity we called a pattern size is a solver rate. Resolved in Phase 4 by measuring the pattern instead of setting it.
3. The shape-to-chemistry feedback	done	One contract, four implementations — curvature, tension, apical area, pressure — because *which* feature the chemistry listens to is the hypothesis, not a dial. Force and size were refused with reasons.
4. A fixture asserting his published constants	done	His Table 1 is now written down in the Grounder, quoted, with the sentence each value comes from — the Hill coefficient, the two diffusivities, the cell cycle, and the two regime *inequalities* that actually define his setting. Asserting a run against it is Phase 3, because it needs the Grounder wired.
5. His cell counts, with the buffer sized for the *destination*	done	**The item that voided the weekend batch**, and it is measured rather than argued — see below.

Phase 3 — Switch on the agents that never run — done (3 of 3)

Ablation is compulsory. A causal claim needs two experiments: adding a mechanism and seeing the shape appear shows it is *sufficient*; taking it away and seeing the shape die shows it is *necessary*. Either alone is an opinion. Every claim now declares its kind and the checker throws out a batch that asks for more than it tests, *before* any simulation starts. Sufficient earns "is sufficient for"; both earn "causes"; neither earns "is associated with". **The word can no longer outrun the experiment.**

The Grounder is wired. The only agent that reads the paper had no call site anywhere — every entry point was exercised by its own smoke test alone. Batch construction now asks it, and it answers with Okuda's numbers carrying the sentence they came from. They arrive as *parameters*, so composition identity is unchanged: setting the starting size is a retune, not a new mechanism. It *advises* — the faithful share of a batch inherits his numbers, the exploratory share is free to leave them, because a search that may only stand where the paper stands cannot discover that the paper is not the only place to stand.

Phase 4 — Measurements that can tell his regimes apart — done

The classifier. Sphere, undulation, tube, branched, plus **invalid** (the surface passes through itself, so no shape reading means anything) and **unclear**, returned deliberately near a boundary. A classifier that always answers can never be wrong, and one that can never be wrong teaches nothing. "Invalid" beats every morphology — this campaign has already once reported a crumpled shell as a result. Until 1 August nothing called it: built, certified, never asked.

The calibration, and the surprise. The plan was to tune until Okuda's handful of spots appeared on a 2000-cell ball, then freeze. Counting spots over time — for the first time — showed the pattern never settles: 104 spots at step 0, 19 at 100, 7 at 200, and **1 by step 400**, where it stays. It keeps merging. So "the right number of spots" is not a setting to be found but **a moment to be chosen**, and the choice has to be stated rather than tuned into.

Phase 4b — Can the chemistry be simulated at all? — done

Why this became a phase mid-flight. The pre-launch stopped on a run whose chemistry turned to infinities within a hundred frames. The rule that exists to prevent exactly this had been **fifty times too generous** for as long as the campaign has existed.

How. The stability condition was written down as a *comment*, worked out once, for the settings of the day. Then the defaults changed, and Phase 2 deliberately widened the ranges so Okuda's own values would be reachable — and the arithmetic in the comment quietly stopped describing the code. On top of that it counted one factor twice, reporting a fiftieth of the true figure. **A comment cannot be re-derived; it can only be re-read, and nobody had cause to re-read it.** The condition is now computed from the settings actually in force, and a composition that breaches it is refused before it reaches a GPU rather than after.

Phase 4c — Multi-spot initialisation, and replication — done

Why. Cedric: *"did you forget to run multiple tubes from now on, not a single one"*. He is right, and the note already said so — one protrusion per run is a single sample, and you cannot tell "this mechanism makes tubes" from "this run made a tube". Several spots give within-run replication for free, and it is what Okuda shows. Every control I built during the pre-launch used one spot or none.

Measured, six runs at the stable point, 300 frames:

seed fraction	act_max	activated fraction	
0.02	0.423	0.40	
0.06	0.501	0.37	three seeds, **identical to 3 d.p.**
0.15	0.598	0.12	
0.30	0.000	0.10	the pattern is extinguished (P4)

Two findings, one of them a problem. Seeding more widely does not give more pattern — it gives *less*, and past 0.15 it gives none at all: a seed covering a third of the shell leaves no room for the inhibitor to separate domains, so the whole field decays together. The useful band is narrow and sits below 0.15.

And the three seeds were bit-identical — 0.501, 0.501, 0.501. `general.seed` was written into every spec and read by *nothing*: both stochastic operators take a `seed` (the mesh jitter, and which cells are nucleated) and the translator passed neither. "Three independent runs" were one run repeated, so every spread this campaign has quoted across seeds described the floating-point reproducibility of a single trajectory.

Fixed, and the fix follows the working specs rather than my first idea. I offset the two operators' seeds, reasoning that jitter and nucleation are independent draws. True, and beside the point: `archive_rounded.py`, `archive_vh_rd_coral.py` and `run_tyssue_fig5.py` all pass *one shared* SEED to both, and a composition that reproduces an archived run has to draw the numbers that run drew. Departing from the convention would have made every reproduction check compare two different experiments. A spec that declares a seed no operator consumes is now refused outright.

Replication, measured for the first time — three seeds, 300 frames, same composition:

	act_max	activated fraction	elongation
before	0.501, 0.501, 0.501	0.374 ×3	1.006 ×3
after	0.501, 0.447, 0.483	0.407, 0.379, 0.374	1.006, 1.006, 1.005
	sd **0.022**	sd **0.015**	sd 0.0005

That last row is what a difference between two compositions now has to beat to mean anything. It did not exist yesterday, and every claim made without it was comparing single samples.

Phase 5 — The first live run: the agents do not talk to each other DONE

It ran, and that is the point. Two rounds, eight simulations, 23 agent calls — and **one usable run, map coverage 0%**. The machinery worked; the science did not, and the reason was the same everywhere.

Not one finding reached another agent. The Critic refused eleven slots and the Proposer was never told why. The Biologist judged every specimen and spoke to nobody. An audit of every hand-off found the same shape **seven more times**: a role produced a correct judgement, wrote it down, and no one read it. *The campaign's defining failure was not bad judgement — it was correct judgement with no recipient.* Drawing the graph is what made it obvious.

Two consequences that are now permanent. Refusals became evidence: a slot the Critic rejects is written to `batch_refusals.jsonl` and lands in the next Proposer's prompt, because a loop told only about its successes cannot avoid repeating its failures. And the Biologist reads `PREMISES.md` mechanically, so the premise gate is a document a person can

argue with rather than a rule buried in code.
The black movie, which is the phase in miniature: frame 0 was perfectly good and was drawn black because frame 800 diverged — the colour scale was normalised over the whole run, so one bad frame erased eight hundred readable ones. Nothing was wrong with the simulation or the plotter alone; they disagreed about what the scale was for.
Closed by Phase 8, which is what this phase was asking for without being able to name it. `WIRING.md` declares every hand-off and `wiring.py` fails the round when an artifact is written and never read — the seven-times-repeated shape becomes a failing check rather than an audit somebody has to remember to run.

Phase 6 — The rebuilt loop, and its first launch DONE

Goal. Rebuild the roster around measurement rather than debate, and launch it.

What the cold start found, and what it cost to misread it

Across all 63 finished runs, `n_protruding` was zero everywhere. **This project has never produced a protrusion.** The loop reported that correctly — and then gave up, which was the expensive mistake, and it was ours. **Track A asks a different question**: a composition that patterns without protruding is a *point on the map*, not a failure. On the corrected ranking, **46 of 63 runs serve Track A**. Only Track B can run out of things worth running; the operator landscape always has more of itself to understand.

Closing Phase 6 — what was run

Nine rounds, 2 August 2026. **79 hypotheses posed, 58 admitted as evidence, 21 refused.** Ten defects found and fixed while it ran — among them: the reading loop recorded *one* run per round and that run was a chimera, assembled from different runs' fields; and the ranking read `premises_broken` from the wrong dictionary, so it never once saw a broken premise.

Results, and what we conclude

The machinery works. 30 predictions were refuted and 23 confirmed — a surprise rate that means the predictions were real rather than safe. **Tubes appeared, on sound specimens. But the ceiling was set in round 3 and never moved.** No valid specimen exceeded `protr_peak` 1.295 or two tubes; every run above two tubes was invalid — dead chemistry or a self-intersecting sheet. And the valid frontier contains **no reaction–diffusion at all**: `cell_rd_seed` and `morphogen_growth_3d`, but neither `cell_diffuse` nor `cell_react`. The shape the campaign can make is not yet the shape the paper is about.

The main conclusion: the logic did not emerge

It was not in the agentic loop, and it was never going to arrive by itself. Nine rounds of capable agents produced conclusions no evidence supported — "the ceiling is FUNDAMENTAL", "division is NECESSARY" — because nothing in the loop could tell an untested possibility from a refuted one, and *could be* had nowhere to live. A campaign that cannot represent "not yet tried" will write "cannot be" instead. That is what Phase 7 exists to fix, and it is a *structural* fix: the unearned conclusion has to be unwritable, not discouraged.

Phase 7 — The agentic loop update: giving the loop a logic DONE

Goal. Supply the reasoning discipline Phase 6 proved will not emerge on its own — as *structure and code*, not as another voice asking the others to be careful. No new agent.

Why one phase and not two. *Absence of evidence read as evidence of absence* and *absence of an instrument read as absence of an effect* are the same sentence with a different subject. Both are cured by the same move: make the unearned conclusion **unwritable**, and give the loop somewhere to put what it has not yet tried.

`LOGIC.md`, and the checker that makes it real. A third source of truth beside the two that already hold — `ROLES.md` kept honest by `roles.py --check`, `PREMISES.md` by the Biologist. `logic.py --check` **refuses a malformed claim at the moment the Meta-review writes it**, so a bad claim never enters the document the Proposer reads. A rule agents are merely *asked* to honour is the failure mode being removed.

Four modalities, and a home for *could be* — which was previously unrepresentable, so a campaign that could not say "not yet tried" wrote "cannot be" instead.

can be	demonstrated positive	one instance suffices
cannot be	universal negative	must carry its quantifier
cannot not be	necessity	the most expensive claim of all
could be	untested, open	**free — and today unrepresentable**

Independence is a distinct condition-signature, not a run count. Ten GM-with-division runs collapse to *one* observation. A negative needs three; **a necessity claim needs three and a failed rescue attempt** — a different experiment, not a re-reading of the same one.

The ablation asymmetry, which is Cedric's and is the sharpest idea in the phase: the same experiment supports a positive strongly and a negative weakly. A null ablation must repeat on an independent background; a positive passes on one.

No conclusion about an unmeasured property. `morphology=sphere` was recorded for the run carrying the finest Turing pattern in the campaign, because the shape was measured and the pattern was not. The honest record is *not measured*, and the honest next step is a request for an instrument — which the Metrologist now builds **on demand rather than on schedule**, so it costs nothing in the rounds that need nothing.

Closing Phase 7

The phase turned out to be wiring, not construction. Almost everything it needed already existed and had never been called: `CLAIM_KINDS`, the batch and memory checks, the reservoir counters, the pattern-scale instrument — already *certified* — the status setter, the allowed-verb table. The external audit's closing line is the phase in one sentence: *"writing it to a file is not the same as handing it over."*

The false start of 3 August. The ceiling cried wolf; the clean start was not clean, and handed a fresh Proposer a memory saying `THIS BODY IS CLOSED`. The lesson is the phase's own turned on itself — a rule written but not wired changes nothing, and that applies to the rules *about* rules. Which is why Phase 8 begins by making the loop runnable with no agent in it at all: the only way to find a broken hand-off without paying for a round.

Phase 8 — Entities, edges and sweeps: the loop stops being a script DONE

Goal. Three things the 3 August runs showed cannot be patched one at a time: give the loop a *declared structure* it can be checked against, close the audit findings that have been open since the graph was drawn, and schedule the parameter sweep that has been implemented and never run.

Why now, and why one phase

Six defects were fixed on 3 August and *not one of them was a logic error*:

`composition.json`	written by `write_config`, never by recon: a producer with no consumer
`run[:14]`	a display truncation read back as an identifier
`batch_ok=None`	a verdict computed, never applied
menu preamble	a contract asserted in prose, enforced nowhere
`max_per_parent=6`	a limit set for one purpose binding another
`say()` vs the wrapper	two owners of one property, folding at 100 and at 96

Every one is an **interface defect**: the components are individually correct and disagree at the seam. That is the class of bug patching cannot retire, because each patch closes one seam and the next round exposes the next. Sixteen more declared edges are unchecked today.

1. Settled by construction: the loop runs with no agent in it

Every defect above was found by *launching*, which is the most expensive instrument available: a round costs ten agent-minutes and half an hour of cluster before it will tell you that a display name was read as an identifier. **The loop must be testable without an agent and without a cluster.** One seam — `run_agent` — carries every model call, and one — `cluster` — carries every job, so both can be substituted for deterministic stubs that return schema-valid answers, including the *wrong* answers we have actually seen: an edit that names a phenotype, a verdict with no flag, a caption with no JSON, a proposal under the wrong key.

A full three-act round then runs offline, in seconds, for nothing, and the questions it answers are exactly the ones that have cost us rounds: does every declared artifact get written, does every gate fire, does the batch fill, is the record complete before its readers run. A regression becomes a failing test rather than a wasted evening. **This is built first, because it is what makes the rest of the phase checkable.**

8.1 — Built, and what it found immediately **DONE**

Three seams, not two. `run_claude` (not `run_agent` — `agents/` is on `sys.path`, so `llm` and `agents.llm` are two module objects for one file, and patching either alone leaves the other live: the first "offline" round spent $0.37 while printing that it was free), the cluster, and — found only by running — **the VLM**. A 23 GB model load is not offline by any reading, and a harness that takes five minutes will not be run before a launch, which is the only thing it is for.

`offline.py` — a full three-act round in **≈6 seconds**, no agent, no GPU, no VLM. `test_offline.py` — **8 cases, 8 passing**, each one a round we have already paid for: the prompt arrives whole; no path reaches the real CLI (proved with a `subprocess.Popen` tripwire); a clean round fills its batch and reaches Act 3; phenotype edits are refused and repaired *in the same round*; a proposal under the wrong key is still read; a reply with no JSON does not fabricate a batch; `REJECT` with no flag is still a rejection; a truncated name resolves to its run.

What it found on its first honest run — the worst defect of the campaign. `run_agent` contained `prompt.split("BREVITY")[0]`, and the Proposer's template pastes `{BREVITY}` at character 331 of a 16,093-character prompt. **The agent received 331 characters**: no legal-move menu, no refusals, no history, no output schema. It invented `add branching` and `add chemotaxis` because it had never been shown the operators — and when it reported *"I lack the injected LEGAL MOVES menu (this call omitted it)"*, that was **literally true** and was read here as a hallucination. Four rounds were spent on the consequences, including the recon Proposer naming zero runs, whose JSON schema sat past character 331 as well. Excising the *block* rather than truncating at it: 331 → 18,025 characters, 36 moves, schema present.

Two faults were the harness's own — worth recording, because inventing failures is the characteristic way a fake goes wrong. Inferring the batch size with a loose regex matched "1 slot" inside a pasted refusal block, so it proposed a batch of one; and a hand-written metrics series omitted `act_mean`, so the Diagnostician read all six runs as diverged and stopped the round over an apparatus fault that did not exist. Both now *template from real files* instead of being written by hand, which is the rule this box exists to state: **a fake that invents failures trains you to ignore the alarm.**

Also landed with it. Same-round repair when the Critic guts a batch, because its refusals are mechanical and have a right answer — measured 1 of 12 → 12 of 12 on round 2's real proposal, one extra call, bounded at one so it cannot loop. Attrition written as a number (`asked/proposed/refused/delivered`) rather than two prints a reader must subtract. `REVISE` reads as a rejection. `[trace]` off by default. And "not checkable" no longer fires on recon slots, which carry no prediction *by design* — it was describing the design as a defect, three lines per run, twelve runs a round.

2. The entities, as classes and a registry

`ROLES.md` already declares who exists and `roles.py -check` keeps it honest — but it checks *existence only*. Each role also declares **Sends to:**, and nothing has ever verified that those fifteen edges correspond to a read anywhere in the code. So a `Role` and an `Artifact` become objects in a registry, each artifact naming its writer, its readers and its schema, and the checker fails the round when:

- an artifact is written and never read — five exist today: `causal_descriptions.md`, `diagnoses.jsonl`, `archivist.jsonl`, `holes.jsonl`, `logic_report.jsonl`

- a declared **Sends to:** edge has no corresponding read
- a prompt asserts a guarantee no checker enforces
- a rendering is assigned to an identifier field

This is the same idiom a fourth time — a document that is the source of truth beside a checker that enforces it — applied to the one thing currently specified only in comments.

8.2 — Built DONE

`WIRING.md` declares every artifact, its writer and its readers; `wiring.py` derives the same graph from the code and reports the difference. **The rule that carries the weight needs no analysis at all: a new artifact must be declared, *with a reader*.** It is deterministic, it is the rule that was broken five times, and it caught `batch_attrition.jsonl` — which I had added an hour earlier and nothing read — within the hour.

The derivation is a corroborating heuristic, and says so. Writes go through class constructors (`BudgetLedger(path=...)`, `Register(path=REGISTER)`) and following those statically needs real dataflow analysis. Two rounds of sharpening — function-scoped windows, then module-constant aliases — took the false alarms from about twenty-five to none, because **a checker that cries wolf is one nobody runs before a launch**. Where analysis cannot reach, the declaration decides; that is exactly how `PREMISES.md` works.

The suite is now 11 cases, 11 passing, including one that exists only to make the checker prove it bites: a row declared with no reader *must* be objected to, or a green tick is indistinguishable from a parser that matched no rows at all.

3. The audit findings, closed

The round record written after its own consumers; the Archivist's `roll_back` truncating at eleven characters against a twelve-character name; the Eye-check defaulting `supports=True`, so an unparsed caption reads as agreement; recon runs that cannot seed the frontier because a verbatim copy carries no composition; and `_dbg_shrink.py`, which does not import.

8.3 — Closed, and one that will not close **DONE**

The eye no longer agrees by default. `j.get("supports", True)` made an unparsed reply — a timeout, a caption the model never produced — read as the Eye-check *agreeing* with the numbers. That is the one direction this role must never fail in: it is the only channel in the loop that looks at *shape* rather than at number, so a silent yes removes the only thing that could contradict the arithmetic, and removes it invisibly. Three states now, and an unheard eye neither supports nor blocks.

The Archivist's identifier is no longer truncated. `comp[:11]` fitted the column and *measurably* collides on the current log — hashes run 12 to 28 characters — and the Archivist names a branch back to the Supervisor *from that table*. A collision there rolls the campaign back to the wrong composition. The region is display and may be cut; the hash is identity and may not. Same disease as `run[:14]`.

Recon runs can be rebuilt into parents — operators, implementations and parameters recovered from the spec, honestly partial about the connections a spec does not carry.

And the finding underneath it, which is not closed. The rebuilt parents are still refused by the Critic: `cell_diffuse0.d_h=2.0` against a declared ceiling of `0.346`, parameters an emitter never receives, `impl: default` for an operator that has no such implementation. **The runs this project has actually made sit outside the space it is searching.** `from_preset` warns of precisely this in its own docstring — *"if a recipe we already trust cannot be built from legal one-edit moves, the campaign is searching a space that does not contain our own evidence"* — and that validation gate is failing. Whether the ranges are wrong or the runs are is **a decision, not a patch**; the loop now says so once per run and falls back rather than pretending.

The harness isolates each round. Sharing the campaign's accumulating state made every successive offline round harder to satisfy — the fourth in a row had every edit refused as a duplicate of its own predecessors. *A test whose result depends on how many times it has been run is not a test.* The real campaign is copied aside and restored at exit.

4. The sweep that was built and never scheduled

`-mode theta` exists, and its discipline is already mechanical: `comp_hash` **excludes** theta, so a retune cannot register as a new mechanism — a change of numbers cannot masquerade as a new idea because the identity function says so, not because an agent was asked to be careful. What is missing is only wiring: `plan()` emits `recon` and `composition` and never `theta`; nothing chooses the base composition, the parameter or the values; and `build_theta_batch` sweeps *one* parameter of *one* composition, so mixtures of parameters are not expressible at all. Track A is not only "does this mechanism matter" but "what does it do as you turn it up", and the second question has never been asked.

8.4 — A parameter change is a move on the menu DONE

Cedric's design, and better than the separate mode it replaces: `set_param` becomes an edit kind beside `add_op`, `remove_op` and `set_impl`, so the Proposer chooses structural and parameter moves from *one* list and can **mix them in a single batch** — instead of a whole round having to be one or the other.

The discipline holds by construction, not by instruction. `comp_hash` already excludes parameters, so a retune yields the *same* composition identity and can never register as a new mechanism. "A change of numbers can never masquerade as a new idea" is true because the identity function says so — asserted as a test, not asked of an agent.

Two points per parameter, taken from each operator's own `(lo, hi, default)`, so a sweep cannot leave the space the Critic admits. **The specimen is not an axis**: substrate parameters, `n_cells` and `seed` are excluded, because `round.py` grounds starting conditions for a reason it learned when two slots ran 500 cells against a control at 2000 — any difference between them confounded the edit with a fourfold change in specimen size.

Structure first, numbers rationed. Parameter moves outnumbered structural ones seventeen to seven on the current frontier, so a menu of any length would have shown mostly dials and a batch would have gone on turning them. Every structural move is offered; parameter moves are capped at a quarter. Track A asks whether a mechanism *matters* before it asks what it does as you turn it up. The preamble now also states what an operator absent from the menu means — *it does not exist in this system, however real the biology* — which is exactly what round 2 needed and was never told.

What you get. A loop whose parts are declared and checked rather than remembered, with the sweep arm switched on. *Nothing relaunches until the checker passes* — and the checker is now `python test_offline.py`: **12 cases, 12 passing, no agent and no GPU**.

Phase 9 — The search could not reach the answer IN PROGRESS

Goal. Six rounds ran cleanly overnight and produced **zero valid protrusions**. An external review, given the campaign and told to verify everything, found why — and it is one missing line, not a hundred strict rules.

The crux, proved by execution

`apply(("add_op", ...))` creates the node and *never the connection*. `is_runnable()` is false for any dangling slot. The Proposer's menu filters on `is_runnable()`. Therefore:

no operator that declares a slot can EVER be added by a one-edit move.

Three operators declare slots, and they are the entire morphogen→mechanics arrow: `morphogen_growth_3d` (`gate`) — *which is Okuda's own mechanism* — `extrude` (`site`), and `divide_3d:orient_iface` (`axis`). The reviewer ran the breadth-first closure of the reachable space:

reachable runnable compositions from the seed	**9,760**
. . . containing `morphogen_growth_3d`	0
. . . containing `extrude`	0
. . . containing **any connection at all**	**0**

`reference_recipes()["okuda_route"]` — the composition this repository has written down as its target, carrying the coupling — is runnable, is admitted by the Critic, and is **unreachable**. The edit set can *destroy* the coupling and can never *build* it: a one-way ratchet toward decoupled compositions.

What that makes of six rounds of findings

The loop's flagship claim is a measurement of its own wiring. `memory.md` records "chemistry is inert for shape — SUPPORTED by every `set_impl` react/diffuse/seed and every chemistry `remove_op`, protr_peak=1.006". Twenty runs returned exactly 1.006, including runs that *deleted the reaction operator outright*. A subsystem with no output edge cannot move a shape, and the loop filed that as a fact about biology.

And the three best results were one artefact seen three times. The top `protr_peak` in the campaign, 2.266, appears on three compositions differing only in their reaction block, with all twenty summary metrics **bit-identical** and byte-identical VLM captions. Its cause: 95% of the vertices collapse to radius 0.08 while a handful fly to 1508, so `protr = r95/rmed` is a ratio of two near-zero numbers. The loop read "identical across three independent bases" as *strength*; it is the signature of a disconnected subsystem. Those three runs also received three different phenotype labels — `exploded`, `spike`, `branching` — from one bit-identical simulation, and those three words are now the whole phenotype inventory of `lever_map.md`.

The sweep arm was dead on arrival. `comp_hash` excludes parameters by design, so a retune cannot register as a new mechanism — exactly the Track A discipline, enforced structurally. But `R6_DUPLICATE` refuses any graph whose hash is already seen, and a `set_param` child has *the parent's hash*. Measured: **39 of 39 parameter moves refused**, even against a seen-list of one. `t_set_param` passes anyway, because it checks that the move is *offered* and never that it is *admitted* — the same interface defect Phase 8 was built to retire, in Phase 8's own test.

The refusal arithmetic, and what it really means. `R6_DUPLICATE` is 106 of 127 refusals; with `DUPLICATE_IN_BATCH`, 95%. But that is the *symptom*: a search that cannot reach new territory keeps re-proposing the territory it has. Rounds 5 and 6 delivered **one run each** — the same control composition that had already run in rounds 2, 3 and 4 — because the control is exempt from the duplicate check and nothing else survived.

Track A's discipline is real, and I was wrong to doubt it. 34 of 46 hypotheses carried a parseable prediction; all twelve `unstated` are round-1 replays where none is owed; of 29 resolved-and-stated, **29 scored confirmed or refuted and 0 inconclusive**. The register works. The content is the problem: 19 of 34 predictions were "nothing will happen", and 18 of 21 confirmations were *the sphere stayed a sphere.*

What Phase 9 does

1. **Make `add_op` of a slotted operator atomic with its `connect`**, when exactly one node produces the required port — the same unique-source rule already used to rebuild parents from specs. This is the one change that opens the space.
2. **Put the reference recipes on the frontier** rather than treating them as a failure fallback. They are the only compositions in the repository that carry a wire.
3. **Scope `R6_DUPLICATE` to structure**, so a parameter move is not refused as a duplicate of the mechanism it perturbs.
4. **The evidence horizon must ask the premise that fired.** `r003c_04` self-intersects at frame 530 — P11 says so — while the record reports `horizon_frame 900`, `valid_frac 1.0`, `valid_evidence True` and a peak measured across the break.

5. **Curiosity as a first-class intent.** Track A asks two questions of a mechanism and the loop can only pose the first. An exploratory slot states what it is varying and what it will *report*, rather than what it expects — "turn χ up and describe what appears" is a legitimate experiment, and a labyrinth nobody predicted is a finding, not a refutation.

6. **The picture must accuse the run.** `metrics.png` plotted twelve of fifty-six available series and omitted both `protr` — the metric the campaign ranks on — and `cells`. Now 3×2: protrusion, cells against time with the array ceiling drawn, mesh faults, activator, the Turing pattern, and the census; with the frame at which the surface stops being a single closed sheet marked in magenta across all six. The camera framed on `r.max()`, so one escaped vertex zoomed a movie out eightyfold and no two runs could be compared; it now frames on a robust percentile, clamped, so a diverged run *clips* instead of shrinking every other run to a dot.

What you get. A search space that contains the answer. Everything this project has built — the specimen gate, the prediction register, the refusal log, the modality logic, the offline harness — has been waiting for one.

Phase 10 — The loop could not say what it was looking at IN PROGRESS

Goal. Cedric, watching a round-2 movie: *"a flash of red activity, 100% red, then a long period of white, no activity."* The loop had no number for that.
It had been running for weeks on a set of measurements that could not tell a real chemical pattern from a uniform wash of the same average brightness, could not say whether the chemistry was still alive at the end of a run, and — as it turned out — was handing the agent that reads each run *nothing at all* about how anything changed over time.

Four faults, all the same shape

Not one is a wrong formula. Each is a thing that was measured and never delivered, or a thing that was asked for and never measured.

An average cannot see a pattern. A field that is uniformly grey and a field that is half bright, half dark have the *same average*. What tells them apart is how much the brightness *varies from cell to cell*: near zero for the wash, large for the pattern. That variation was never measured. The loop could only see the brightest cell (`act_max`), which reads high both for a healthy pattern and for a single cell blowing up, so a pattern that spiked and a pattern that died looked identical.
On the end state of the run named `okuda_route`, the activator varies by 8.4×10^{-5} across 3,975 cells around an average of 0.0128. The chemistry is dead. Every number the loop was allowed to quote stayed perfectly sayable while it died.

The measurement that answers the campaign's own question was never taken. Everything here turns on one thing: *does the chemistry drive the shape, or is it decoration on a shape made by something else?* The number for that — how strongly a cell's chemical level tracks how far out it sits (`corr_act_rad`) — was on the list of things an agent was allowed to predict, and was computed *nowhere in the codebase*. A prediction naming it scored "not measured", quietly became "inconclusive", and no warning was printed anywhere. The agent wrote a perfectly good falsifiable claim and the loop recorded that it had learned nothing. Two other shape measurements were in the same state.

Nothing about time was reaching the reader — and had not for the whole campaign. The per-frame table stores each frame's shape verdict as a *word* ("sphere", "tube"). The code that classifies how each measurement behaves over time tried to convert every column to numbers, including that one, and crashed on the word. The crash was caught one level up and turned into a single parenthesis in the reading agent's prompt: *"(trajectory shapes unavailable)"*.
So since the day that column was added, **every run has been read with no history at all**: no warning that a run peaked early and decayed, none that it was still climbing when the clock ran out, none that it was pinned against a buffer, and no evidence horizon. The curves were computed, drawn into a figure, and classified — and the classification died on the one column that was never meant to be classified.

An instrument that was withdrawn for lying was still being advertised. One measure of pattern wavelength was taken off the approved list months ago because it had never been calibrated. The name stayed on the list, and worse, it was in the short banner that tells every agent *"these are new, use them"*. It has been renamed to say what it is and marked as untrustworthy.

What was built

- **The pattern, as numbers.** How much the chemical varies across cells (`act_sd`); the same divided by the average (`act_cv`), which does not care whether the overall level is collapsing or exploding, so it still means something when the chemistry is running away; and what fraction of the tissue is switched on (`act_occupancy`). A frame counts as *patterned* only if *both* the variation and the occupancy are real — so one cell of four thousand blowing up, which makes the variation enormous but the occupancy 0.01, is correctly not a pattern. Over a whole run: what fraction of it had a pattern at all, and the frame at which the pattern last died.
- **A correlation is refused when there is nothing to correlate.** The chemistry-drives-shape number reads a confident 0.294 on that dead field — a correlation of rounding error, because the statistic is deliberately blind to scale and will happily report on noise. It is now simply *not measured* when the pattern is below the liveness threshold, which makes "does the pattern drive the shape" come out *inconclusive* when there is no pattern. That is the honest answer. Given a planted signal — chemical painted onto the outermost 15% of cells — the instrument recovers it: 0.724, and the tip-enrichment measure reads 5.19 against a null of 1.0.
- **Telling a tube from a lumpy ball.** The elongation number in use is a ratio between two points of the radius distribution, so it reads the same for one long tube and for a broad bumpy sphere. The shape of the whole cell cloud — long and thin, round, or flattened — now has its own three numbers. This is Okuda's own axis: tube, undulation, branch.
- **The history channel, revived and widened.** Word-valued columns are skipped instead of crashing. The list of curves allowed into a prompt held seven entries and *not one about the chemistry*; it now covers shape, cells, pattern and apparatus. And each curve carries eight numbers across the run beside its one-word verdict — because "peaked" describes a gentle rise and fall *and* a spike to seventeen thousand followed by extinction, and only the numbers say which. That run now reads `0 0 0 0 1 0.013 0 0` for the fraction of switched-on cells: the flash Cedric saw, written down.
- **Three checks so this cannot recur quietly.** Every approved measurement must have something that actually computes it; every one must have a written definition, and no

definition may name a withdrawn measurement; and the history channel must survive a word-valued column. Twenty-one checks pass.

One name, one quantity

A **plain name** means a per-frame measurement: computed every sampled frame, classified over time, and reported at the last frame so a prediction can be scored against it. A **suffix** is only used when the number is *not* simply the last frame, and says what it is instead — the best value reached, or the last *trustworthy* frame before the mesh tore.
Thirteen duplicate names were deleted under that rule. They had been created by copying every per-frame measurement into the summary twice, so the same number appeared under two names. A set of measurements is a *vocabulary*: doubling it without adding a quantity makes it harder to use and measures nothing new. The approved list went from 53 names to 40.

Grouped by the question they answer

In round 2, all twelve predictions were written about elongation while every chemistry measurement sat unnamed. The approved list had reached the agent as one comma-separated line of fifty-three identifiers, and a flat list is a list you read the first entry of.
Every approved measurement now sits in exactly one of five groups, phrased as questions: *is it a tube; is it still made of cells; is there a pattern at all; does the pattern grip the shape; is this evidence at all.* Each is printed with the value it takes when the answer is *no* — because a prediction against a bare identifier is a guess. "Variation above 0.3" means nothing until you know a dead field reads 0.00 and a live one reads about 1. The written definitions went from 6 to 38.

How often to measure

The old rule gave *every* run forty samples, however long it was. A 900-frame run was therefore sampled every 23 frames and a 300-frame run every 8 — so "the twelfth sample" meant a different moment in each, and two runs could not be compared point by point, which is the one thing a history is for.
It is now every 25 frames, always, with the last frame added. Measured cost: one frame of analysis takes 1.43 seconds on a real 3,975-cell mesh, so

sampling	frames analysed	cost per run
old rule (forty samples)	40	57 s
every 25 frames	37	**53 s**
every 10 frames	91	130 s
every frame	901	21.4 min

The new rule is *cheaper* than the one it replaces. Measuring every 10 frames would give two and a half times the resolution in time for about 77 extra seconds a run — roughly 15 minutes on a twelve-run round — which is the trade to make if "when did the pattern die" needs answering to within ten frames rather than twenty-five.

What is still open

1. **Which measurements to keep.** The per-frame table produces 64 numbers and much of it is one quantity written twice — three spellings of "fraction of switched-on cells",

a count and a fraction of the same fault, and two different ways of asking whether the chemical sits at the tips, one of which was added this week without noticing the other. A cut to 24 is drafted and is being checked against every consumer — a figure panel, a biological premise, a shape classifier — before anything is deleted. One redundancy is exact and confirmed; another that looked exact is wrong by 1% at the precision stored, so that measurement stays.

2. **What to say about time.** With a single per-frame table at a fixed spacing, separate measurements for start, middle and end stop being necessary: they become *summaries* of one history — last value, best value, average, spread, trend. Which summaries are worth their space in a prompt carrying 24 measurements is the open design question.

3. **Telling the agents.** Approving a measurement makes a prediction about it *scorable*; a separate short list is what tells an agent the measurement *exists*. The new chemistry measurements were approved and not announced, so round 2 wrote every prediction about elongation while they were computed for the first time and named by nobody. The same fault as the others, one list over.

What you get. A loop that can state what it is looking at. The reviewing agent's objection to round 2 — that five of its twelve experiments were reading a shape-to-chemistry feedback off a field that was already dead — was, until this phase, an assertion no instrument could settle. It is now a number.

Phase 11 — An outside reviewer says no, and why — IN PROGRESS

Goal. The loop ran for the first time with the metric work in place, and I asked three independent reviewers to judge it against Track A — read-only, verify by execution, quantify or it does not count. The verdict is *no*, and it names the sentence: *"what does this mechanism do as you turn it up" is the second question Track A asks of every mechanism.* That question was asked **zero times in 36 slots**, and not by omission.

The number I reported was wrong

I told Cedric the campaign "reached best `protr_peak` 2.338, a sphere being 1.0". It did not. 2.338 is the *composite score* $0.5\,\texttt{protr\_peak} + Q + 0.25\min(\texttt{n\_tubes}, 2)$, and it is a **rail**: eleven runs across ten distinct compositions hold it to four significant figures, every one with `protr_peak` = 1.022. The best protrusion the campaign measured is 1.185 on an exploded mesh, and 1.004 on a premise-clean specimen. **There is no protrusion in this campaign.** The cluster freeze — "no gain in 13 evaluations" — was the loop reading its own rail as an exhausted idea.

The four faults, each with the number that makes it one

A sweep was arithmetically impossible. `comp_hash` is deliberately parameter-blind so a retune cannot be filed as a new mechanism, and it works: **107 of 107** parameter perturbations leave the hash bit-identical. But `round.py` deduped slots *within* a batch on that same bare hash, while `proposer.py` *requires* slot 0 to be the unchanged parent — so every parameter move collided with the mandatory control. **48 of 48** admitted sweep slots were refused `DUPLICATE_IN_BATCH`, and the refusal text "Vary the EDIT, not the number" then entered the next Proposer's prompt as a lesson. The loop was teaching itself not to ask Track A's second question. Measured on the live frontier: the 25 sweep edits available share *one* composition hash and have **25 distinct run keys**. `critic.py` already keyed sweeps on the run key — mechanism *and* operating point — one screen away. Two readings of one

identity function disagreed about what a sweep is, and the stricter one won by being later in the file.

Predictions were scored on specimens the same record called invalid. The Biologist writes `premises_broken`; `check_posthoc` never read it. So a run that passed through itself (P11) or absorbed area by stretching (P7) was scored `confirmed` in `hypotheses.jsonl` while `collector.py` wrote "specimen invalid … describes the configuration and not a tissue" about the *same* run into `analysis.md`. Two records of one experiment disagreed, and the Supervisor reads the register — so the surprise rate, the 70/30 steer and the cluster freeze were all computed over runs the Biologist had already rejected. **24 of 35** archived runs are refused once the verdict is wired to the resolver.

The loop could not be curious. `INTENTS` held `confirmatory | adversarial | control`. `Hypothesis(intent="exploratory")` raised `ValueError`, and so did `predicted=""`. So every slot had to be a bet, and the trap the goal names — *a loop that may only test predictions will only ever propose things it can predict* — was structural, not cultural.

Nine of twelve runs were the same mesh under nine names. Both frontier parents carry `conns: []` and no growth operator, so the chemistry cannot reach the mechanics and the shape is a *constant function* of the edits being made. Re-measured here: **10 of 12** round-3 runs share `gyr_prolate_peak` to sixteen digits across ten distinct compositions. Seven of them resolved `confirmed`.

What the reviewers said is working, and should not be touched

Not everything is broken, and a review that finds only faults gives no basis for deciding what to keep. The prediction record is genuinely prospective and append-only: 36 posed before the run, 36 resolved after it, none posed-and-never-resolved, and `resolve()` raises on a second write. The evidence horizon was called "the best instrument in the repository" — it refused 86% of one run's frames because the mesh self-intersects from frame 100. The premise battery is doing real work: P11 catches self-intersection that the genus check reports as a clean sphere. And the metric bank fix landed: the assembled Proposer prompt is 33,701 characters with the admitted list in it, and the batch went from twelve predictions on one metric to **eleven distinct metrics**, with `act_cv_peak` — the pattern-existence test that did not exist the day before — the second most used.

And the fault none of them was asked about

Ten of the fifteen runs were refused as `P3_CHEMISTRY_DIVERGED`, `act_max_peak` $= 9.51 \times 10^5$. Two thirds of the GPU budget bought no evidence, and the loop read the refusals as facts about the mechanisms it was testing.

The cause is one unset parameter. Gierer–Meinhardt's saturation — $a^2/(h(1+\kappa a^2))$, whose own comment says it "bounds the activator peak so self-enhancement can't run away" — **defaults to** $\kappa = 0$. `okuda_route`, the supplied reference recipe that the whole search breeds from, never set it, and `sat` was not in the tunable table either, so the loop could neither set it nor sweep it and every child inherited the divergence. Measured over 250 frames:

κ	act_max_peak	act_cv_peak	alive_frac
0.0	9.51×10^5	0.188	0.052
0.02	2122	3.945	0.311
0.1	**1489**	**4.138**	**0.375**
0.5	599	2.802	0.307

0.1 is the measured knee: harder saturation caps the peak further and weakens the pattern. It is now the recipe's operating point *and* a tunable parameter, so the loop can ask what the saturation does rather than inheriting one value forever.

coral_gate — two half-answers in the archive

Cedric: *"fix okuda_route with a chemistry that works, see specs that works in the archive"*. Surveying all 50 archived runs, 21 have a live and bounded chemistry, and every one of them uses **Gray–Scott**, not Gierer–Meinhardt. Two half-answers sat side by side:

- `r002c_09_a23094` carries the *full* Okuda coupling — `cell_react` → `morphogen_growth_3d` on the `gate` slot, plus `shape_to_chem:curvature` — and reached `protr_peak` 1.185 over all 900 frames at `valid_frac` 1.0. Its chemistry is dead (`act_cv_peak` 1.9×10^{-5}), so that shape was pure mechanical buckling with the growth gate closed.
- `coral_fixed_ball` has the opposite: Gray–Scott, alive and bounded (`act_max` 0.46–0.76), and *no coupling at all*, so it stays a sphere at 1.006.

`coral_gate` is the structure of the first with the chemistry of the second, and three separate checks each caught something the others could not:

1. **The Biologist refused it before any GPU time.** I had kept `rho: 0.0` so growth would be purely gated, reasoning that a baseline would grow the sphere uniformly and mask the mechanism. P2: *"an ungated cell grows at EXACTLY zero and the tip/body growth ratio is infinite. No tissue does that."* It is the better argument — a gate that shuts growth off is a morphogen deciding *whether*, which is the premise's name — and the masking I feared is measurable rather than hidden.
2. **The new metrics caught the second.** `act_max_peak` = 0.0, `n_spots` = 0: the Gray–Scott field was never started. Its $u = 1, v = 0$ state is a *stable* fixed point, and Gierer–Meinhardt's self-starting `a0` baseline had gone with the swap. The run still reached `protr` 1.096 and "undulation" on mechanics alone — a convincing movie of nothing.
3. **The missing piece was an operator, not a number.** `cell_rd_seed`, present in coral's spec and absent from the archived one. Each half-answer lacked exactly what the other had.

With all three: `act_max_peak` 0.789 (bounded), `act_cv_peak` 2.20, `act_alive_frac` **1.00** — patterned for the whole run — **100 spots**, `corr_act_rad` measurable on every frame at 0.435, morphology *branched*, and `valid_frac` 1.0. All ten premises hold.

What is in place for the next launch

- the sweep arm: 1 admitted slot of 25 → 25;
- the specimen verdict gates the resolver, so the two records cannot disagree;

- `exploratory` intent, resolving `described`, outside the surprise arithmetic and still required to state what it varies and what it will report;
- a null-difference gate (per slot) and a decoupled-round gate (per batch);
- `coral_gate` in the starting pool in place of coral; `okuda_route` removed at Cedric's instruction and shelved under `_keep/`;
- `sat` fixed on the reference recipe and made tunable.

What you get. A loop that can ask Track A's second question, cannot score a broken specimen, and starts from a composition whose chemistry is alive. What it still does not have is a protrusion.

Phase 12 — The reduction DONE

Goal. Take the one-agent loop as the control, and cut this one down to what composition actually requires. Not an addition. Cedric, 5 August: *"my feeling is that we should not add at this point but remove, loose, look for a simplification."*

The control: 305 lines, one agent, one call

`GNN_LLM.py` and `GNN_LLM_task.py` in `connectome-gnn-cx` run a discovery loop that has produced usable science for months. Together they are **305 lines**. One agent, one call per round, a free-text `analysis.md` and `knowledge.md` that the same agent writes and re-reads. No critic, no register, no premises, no intents, no budget ledger, no eye.
Against it, measured this morning:

	one-agent	discovery_okuda
Python driving the loop	305	23,725
distinct agent roles	1	15
LLM calls in the last round	1	**153**
LLM minutes in the last round	∼2	**166.6**
state files	2	22

A factor of 78 in code and 153 in calls. The last round of the expensive one produced **twelve refusals and a rollback**. That comparison is the whole argument of this phase.

What multi-agent genuinely buys, and it is one thing

Cedric, 5 August: *"the one agent has not much but do not composition."* That is exactly the line. The one-agent loop tunes the parameters of a *fixed* model — the graph, the buffers and the operators never change, so a proposal cannot be structurally invalid and nothing needs to check it. This loop *composes*: it chooses which operators exist, how they wire, and how big the reservoirs are. That creates failure modes the control cannot have, and they are the only ones worth a gate:

- a spec that does not compile, or wires a slot that the implementation does not expose;
- a reservoir too small to reach the run's own target — the arithmetic that voided **59 runs** in two batches, every one ending at exactly 1778 cells;
- a composition already evaluated, spending a GPU on a duplicate.

Those are *structural*, cheap, and decidable before a GPU is touched. Everything else this loop refuses, it refuses on *judgement about a result* — and that is where it went wrong.

Premises: an input, not a gate

Cedric, 5 August: *"I like the premise.md but as an input not a gate."* The premise battery produces the best diagnostic text in the system — *"volume went 522.1 → 312.9"*, *"the top 5% of cells reach shape index 5.83"*, *"the surface folds through itself at frame 50"* — and every word of it was spent on a **refusal**. I wired it into the resolver on 4 August and it stopped the campaign: P1, P7 and P11 fail on essentially every composition-mode run, so 12 of 12 were refused in two consecutive rounds, both aborted, and the loop halted itself. Its own message was correct: *"the refusal reasons are not actionable."*
There is a clean division of labour the gate was blurring, and keeping it makes the removal safe:

- **the evidence horizon** decides which *frames* are trustworthy. Arithmetic, per frame, already truncating everything measured after the mesh tears. *This is the mechanism that stops a number being believed, and it is untouched.*
- **the premises** say what is biologically wrong, in words carrying numbers. Their consumer is *the next proposal.*

The audit's actual finding was narrower than I read it: `hypotheses.jsonl` said `confirmed` while `analysis.md` said *specimen invalid* about the same run — two records *disagreeing.* The fix is to write the verdict beside the outcome (`confirmed [specimen: P1, P7, P11 broken]`) so a reader can weigh it. Refusing the run instead threw away the diagnosis along with the evidence.

The gates that go, and the ones that stay

stays	`R1-R4`, `R6`, `C1`, `C2`, `C3`	structural: can this be built, is it new, can it reach its target. What composition requires and the control cannot need.
goes	`P0_SPECIMEN_INVALID`	fired on 12 of 12 runs. A gate that fires on everything carries no information and costs the round.
goes	`P1_INERT_OPERATOR`	"`cell_diffuse` did nothing at $D = 0.002$" is *knowledge.* Refusing the run deletes it.
goes	`P3_CHEMISTRY_DIVERGED`	nan cannot be scored confirmed or refuted anyway — the arithmetic refuses itself, so the gate was ceremony.
goes	`check_null_difference`, `check_round_decoupled`	written 4 August, called only from the test file. Orphans.

Four roles. The eye is one of them.

Proposer	unchanged, and now shown the diagnosis. 59% of the budget (98.2 of 166.6 min, 44 calls) — the expensive role, and the one doing the science.
Analyst	reader + interpreter + meta_review + collector + diagnostician. One call over the whole batch, which is the control's shape: the agent that reads the round is the agent that decides what it meant.
Eye	*kept as its own role.*
Grounder	demoted to code that speaks only when something changes.

Why the eye survives a phase whose purpose is removal. Cedric, 5 August: *"I like the eye I think it is valuable."* It earns it on the record. On **2 of 10** runs last round the Reader wrote `phenotype sphere` and the metric agreed with it, while the eye, watching the same movie, wrote *"develops large protrusions and irregular lobes"* (`r001n_07`, `protr_peak` 1.10) and *"transforms into an asymmetrical, bulging, elongated form"* (`r001n_10`, 1.26). It also flagged, unprompted, that *"the embedded circular cross-section appears to be a measurement artifact rather than tissue feature"* — a rendering bug no metric was watching for.
So it is not a redundant judge, it is a *separate instrument*: a VLM on the movie, a capability no text role can substitute for, disagreeing with both the metric and the Reader on a fifth of the batch. And it is the *cheapest* role in the loop — 26 calls, 6.2 minutes, **3.7%** of the round. Removing it would save 4% and cost the only channel that looks at the picture.
The eleven roles that go were not badly designed. They were badly *wired*: each produced text that the next one was not shown. `grounder.py` is the clearest case — **766 lines, 16 public functions, 4 of them ever called**. The two that would answer the question Cedric asked weeks ago (*compare the results to Okuda*) are `gate()` and `figure_target()`, and they are called **zero** times. Its repeated ϕ note demands a decision — *"must be made explicitly, not inherited"* — that no role can take, so it reprints identically every round.

What this removes

orphaned code (written, never called)	1,690
eleven roles' prompts, parsers, state	∼2,900
post-hoc gates and their plumbing	∼190
removable	**∼4,600**
irreducible substrate (operators, engine, metrics)	∼7,600

`round.py` rewritten from scratch

Cedric, 5 August: *"should we start round from scratch — basically it has to write four prompts, collect the answer and launch the simulations . . . but round.py if it is well written I'm not sure it should end up in tons of lines."*
It should not, and it does. `round.py` is **2,504 lines — 1,605 code, 683 comment**, and two functions hold most of it: `_run_round` at **657 lines** and `_admit_slots` at **267**. It also carries three batch modes, of which `theta` has **never run once** in six rounds (4 composition, 2 recon).
The loop has nine things to do. Everything else in that file exists to manage machinery that this phase deletes.

#	step	now	rewritten
1	read the parent set	71	∼15
2	assemble one prompt: parents, metric bank, last round's diagnosis, legal menu	scattered	∼40
3	**one** Proposer call → slots	—	∼15
4	per slot: parent + one edit → emit → `critic.admit` → yaml	428	∼60
5	launch	*unchanged*	
6	read metrics per run	*unchanged*	
7	**one** Analyst call over the batch → `analysis.md`, `knowledge.md`	5 roles	∼40
8	the eye on each movie	*unchanged*	
9	append one record row per run	22 files	∼25
	total	**2,504**	**481**

Metrics, operators, `translate.py` and the engine are untouched. Those are the substrate and they have earned their size; the loop that drives them has not.

The largest single cut is step 4, and it is also the budget fix. `_admit_slots` retries, re-asks, dedupes across the batch and runs a "repair pass" — which is why the Proposer made **44 calls for 12 slots** and spent 98 of the round's 167 minutes. Rewritten, it admits what is legal, prints what was not, and runs the short batch. One call. That removes the 59% budget line and ∼500 lines with the same edit, and it is the same lesson as the gates: the machinery was there to rescue a judgement that did not need making.

Deleted along with it, because all of it exists to manage that retry loop: `run_escalation`, `quarantine_scan`, `stale_q_reason`, `_abort`, `_refusal_summary`, `_last_review`, `build_theta_batch`, `_resize_reservoir`.

Four roles, four scripts, four markdown files

Cedric, 5 August: *"since we have four agents I want four script, four md file use to refine their role?"*

This is the control's actual secret, and I had missed it. The one-agent loop's knowledge does not live in `GNN_LLM.py` — 305 lines that barely mention biology. It lives in `instruction_cortex_matrix.md`, **279 lines** of plain markdown that Cedric edits between rounds. Months of accumulated judgement about what to try and what a result means sits in a file that needs no Python to change. Ours is the opposite: **∼418 lines of prompt text** are welded into f-strings inside `proposer.py` (172), `llm_agents.py` (185) and `llm.py` (61), where refining a role means editing code and re-reading a docstring to find out what the role was told.

So: one script and one markdown file per role, and nothing else in `agents/`.

`proposer.py`	`proposer.md`	what to try next, and what makes a good experiment
`analyst.py`	`analyst.md`	how to read a batch: metrics, the eye's captions, the premise diagnosis; what to write to `knowledge.md`
`eye.py`	`eye.md`	what to look for in a movie, and to say plainly when the picture contradicts the number
`grounder.py`	`grounder.md`	Okuda's figures and settings, and the comparison of *this* round's result against them

The split is the point. The `.md` is the *role* — stable, in the user's hands, versioned in git, edited between rounds without a Python change. The `.py` is only *plumbing*: gather the data, inject it, call, parse, validate. A prompt built as `md + injected data` also makes the prompt readable as a file, which no amount of docstring achieves.
And this is what makes the Grounder the fourth agent rather than dead code. It was never badly designed — 766 lines, 16 functions, 4 ever called, and the two that would compare a result to the paper (`gate()`, `figure_target()`) called **zero** times. Its repeated ϕ note is a decision demanded of nobody, printed forever. Moved to Act 3 with its knowledge in `grounder.md`, it answers the question Cedric asked weeks ago and the loop has never once asked: *does this look like Okuda's figure?* When it is wrong or repetitive, the fix is to edit 279 lines of markdown, not to re-wire 766 lines of Python.

The engine is blind to the roles

Cedric, 5 August: *"can we have the round engine kind of blind to the role of the agents?"* Yes — and this is the structural cause of the 657-line function, not merely a tidier design. `_run_round` grew because it *knows every role by name*: a retry loop written for the Proposer, a budget carve-out for the Reader, a repair pass for one failure mode, an escalation path for another. Every role that was added arrived with a special case in the engine. **An engine that cannot name a role cannot grow one.**
So the engine knows three *stages* and no roles at all:

stage	fan-out	the stage's contract
`propose`	one call	returns a list of edits
`observe`	per run	returns text about one run
`review`	one call	returns text about the batch

Each role is a directory entry, not a branch in the engine. Its `.py` declares where it attaches and what it needs; its `.md` is the prose:

```
ROLE = {"stage": "review", "wants": ["metrics", "observations", "history"],
        "writes": "grounding.md", "md": "grounder.md"}

def run(bundle):  ...
```

and the engine is the whole of this:

```
for a in roles_at("propose"): edits = a.run(bundle)
specs = [build(e) for e in edits]          # emit, critic.admit, write yaml
runs  = launch(specs)                      # the engine's own work
data  = [measure(r) for r in runs]         # metrics module, unchanged
for a in roles_at("observe"):
    for r in runs: a.run(bundle | {"run": r})
for a in roles_at("review"): a.run(bundle | {"metrics": data})
```

What this buys, concretely. Adding or removing an agent is adding or removing two files. Dropping the eye, or adding a fifth role next month, requires *no edit to the engine* — which is the test this design has to pass, because the eleven roles removed in this phase were each added by editing the engine. The words `proposer`, `analyst`, `eye` and `grounder` should not appear in `round.py` at all, and that is checkable in one grep.

The one piece of role knowledge that cannot be removed, stated honestly. Something must return *edits*, because the engine has to build a spec from them. That knowledge belongs to the *stage contract* — `propose` means "returns edits" — and not to any role, so any file may occupy that stage and a round with nothing in it is simply a no-op rather than a crash. The same holds for `observe` and `review`: they return text, the engine files it, and the engine never inspects what it says. A role that produces something no stage consumes is then impossible to write, which is the defect this campaign hit six times.

And an instruction file for the round

Cedric, 5 August: *"and an instruction file for round?"*
Yes, and it half exists already — conflated. `campaign/instruction.md` is 63 lines, tracked in git, and its third section reads *"## What you are — You are the PROPOSER."* The *campaign's* objective and *one role's* identity live in the same file. That is why the Proposer is the only role that has ever had standing instructions a human could edit: the campaign file *was* its prompt, and the other fourteen roles had nothing but f-strings.
Split into two layers, five files:

`round.md`	**shown to every role.** The objective; the discipline (a change of numbers is not a hypothesis; every candidate carries a falsifiable prediction; ∼70/30 confirmatory to adversarial); what is already known; what has been ruled out and must not be re-proposed.
`proposer.md`, `analyst.md`, `eye.md`, `grounder.md`	**shown to one role.** How that role does its job, and nothing about the campaign.

Every prompt is then `round.md` + `<role>.md` + injected data, assembled by the engine in that order. The engine *reads* both files and interprets neither, which keeps it blind: it cannot behave differently for one role because it never learns which role it is holding.
One line I will not cross: the round's numbers stay in config, not in markdown. Slots per round, the control slot, the launch budget, the analysis stride and the stop rule are values the engine must *obey*; a number the engine has to parse out of prose is a number that can silently drift from the number that ran. `round.md` carries the *procedure and the judgement*, and `config` carries the *quantities* — the same division that makes the metric bank trustworthy.
Where the knowledge accumulates, which is the whole reason the control works. The Analyst writes `knowledge.md` each round; you fold what survives into `round.md` between rounds. That is precisely the cycle running in `connectome-gnn-cx`, where 279 lines of hand-edited markdown are carrying months of judgement, and it is the one mechanism this loop has never had: fifteen roles, and not one file a human could edit to make the next round smarter.

The round is a graph, and the engine only runs it

Cedric, 5 August: *"what about a graph provided to the round with agents and information flow so that it is blind … that would be easy to modify, easy to understand instead of hardcoding?"*
Better than the three hard-coded stages I had built an hour earlier, for a reason specific to this campaign. In the stage version each role declared a `wants` list — and the engine passed the whole context regardless, so the declaration was decoration. Nothing could check it. As a graph the information flow *is* the artefact, so **the defect this loop hit at least six times**

becomes arithmetic: a node that emits something no `in:` names is refused at load, before a GPU or a token is spent.
`crew/flow.yaml` is 120 lines and is the whole multi-agent design — not a diagram of it:

```
- id: diagnosis            # why last round's tissue broke, ranked
  code: engine.diagnosis
  in: [parents]

- id: proposer
  agent: proposer
  in: [parents, menu, metric_bank, diagnosis, history]
  out: edits

- id: eye                  # one call per run, on the movie
  agent: eye
  in: [metrics]
  each: names
  out: observed

- id: analyst
  agent: analyst
  in: [control, metrics, predictions, observations, observed, history]
```

Seventeen nodes: thirteen `code:` steps and the four `agent:` roles. The engine loads it, checks it, sorts it topologically and executes it, and holds no opinion about what any node is for. `engine.py` is **481 lines** and `test_engine.py::blind` asserts that no role's name appears in its executable code, so the assertion fails the moment a special case is written for one of them.
Four checks at load, and the third is why the graph earns its indirection.

every `in:` is emitted	otherwise a role runs on missing data and says something confident about nothing
no cycles	arithmetic
every emission is consumed	unless the node is terminal. `steer` never reached the Proposer; the premise diagnosis was spent on a refusal; `sat` was set in two places and emitted in none; the eye disagreed with the Reader on 2 of 10 runs and nothing compared them. All found by hand, weeks later.
targets resolve	a typo in the flow is not a silent no-op

The check fired on the flow's first load and on me twice. First: *"'record' emits 'record' and NO node consumes it"* — correct, and the honest answer was that its output is a file, so a terminal node now declares `writes:` rather than being exempted by the engine guessing. Second, while writing the test that drops the eye: deleting its node left `observed` in two roles' `in:` lists, and the loader refused with *"'analyst' needs 'observed', which no node emits."* That is exactly the half-finished edit a human makes, so the test now asserts both halves — the incomplete removal is refused, the complete one loads.

What was built, and what it measures

`engine.py`	481	load, check, sort, execute; thirteen code nodes
`crew/*.py`	453	four roles, ∼65 lines each
`crew/flow.yaml`	120	**the design**
`round.md`	84	the campaign, shown to every role
`crew/*.md`	233	four roles' judgement, yours to edit
`test_engine.py`	232	88 checks, all passing

Against `round.py`'s 2,504. **Measured in executable statements** — the fair comparison, since the old file explains in 683 `#` comment lines and the new ones explain in docstrings that a line counter scores as code — it is **437 against 1,099, a factor of 2.5**, with the wiring moved out of code entirely. My earlier estimate of ∼350 lines for the engine was low by a third: the flow loader and its four checks are real code that the hard-coded version did not need.

Both answers, as decided. `recon` is kept, and in the graph it is not even a branch — it changes one argument in the `launch` node. The new files sit beside the old: `round.py` is untouched and still runs the campaign, and two of its functions are imported rather than copied (rebuilding a graph from a finished run's spec, reading a diag summary — 240 lines of real logic) until it is deleted after one clean live round.

The metrics: one class each, and a registry

Cedric, 5 August: *"I remember we have a lot of metrics — can we create a class for each of them, instead of a distributed code? add a registry to structure these classes. point the metrics we gonna use in the md file of the analyst?"* And: *"the work on metrics should go into phase 12 simplification."*

It belongs here because it is the same disease as the flow, one level down. Adding *one* metric today means editing **six places**, and none of them is next to any other:

computed	`prototype/Tyssue/tube_analysis.py`
reduced over time	`run_one.py` — `_final _peak _floor _trend _span _measured_frac`
admitted	`predict.SERIES_QUANTITIES` (24) × `SUFFIXES` (6) = 156 names
documented	`predict.METRIC_NOTES` (32)
grouped by question	`predict.METRIC_GROUP` (5)
withdrawn / rejected	`predict.WITHDRAWN` (114) / `REJECTED_METRICS` (4)

And then consumed everywhere: `protr` alone appears in **eight** files — biologist, critic, control, collector, archivist, logic, engine, composition_space. `act_cv`, which I added two days ago, is in eight. A quantity whose definition is in one file and whose *meaning* is in another is exactly how `spot_spacing_cells` came to be admitted in three suffixed forms that no real run's summary contains — so a prediction naming it can never be scored, and that is the one failure still open in the old test suite.

```
@register
class ActCV(Metric):
    name   = "act_cv"
```

```
    group  = "chemistry"                  # which question it answers
    note   = "activity CV -- scale-free, survives a collapsing or exploding level"
    series = True                         # -> the six temporal reductions
    refuse_if = "act_cv <= 0.05"          # the guard that is inline today
    def compute(self, f):
        act = f.activity
        return act.std() / abs(act.mean()) if abs(act.mean()) > 1e-12 else 0.0
```

One class replaces six registrations, and the registry *derives* what is hand-maintained now: `registry.names()` for the 156 admitted names, `registry.notes()` for the bank the Proposer is handed, the `group` field for `METRIC_GROUP`, `@withdrawn("reason")` for the 114, `registry.compute_frame(state)` for the `frame_metrics` monolith, and `refuse_if` for the guards written inline — the one that says `corr_act_rad` must not be computed on a dead field.

On pointing the metrics from `analyst.md`, with one change. The md should name the *questions*, not the list. If it carried the 156 names they would drift from the registry, and a metric list that has stopped describing the code is this campaign's most reliable defect — a limit in a comment, a paper's own ϕ, a rail read as a result. So `analyst.md` says *"lead with protrusion: `protr_peak`, `n_tubes`, `protrusion_aspect_max`"* and the registry stays the source of truth, with a test asserting that **every metric named in any `.md` exists in the registry**. Prose may point; it may never define. Same rule as the flow check, and the same reason.

Not while a round is live. `cluster.py` maps `/workspace` to `/groups/.../Graph`, and `run_one.py` — which executes *on the cluster* — imports `predict` for `SERIES_QUANTITIES`. So the entire metric stack is live code for the seven jobs now at frame 110 of 900. Editing it mid-round would reproduce the defect that broke the Table 1 reproduction exactly: trained at one commit, evaluated at another, with the metric redefined in between. The refactor waits for round 3 to land.

Built, and what it measures

Committed as `dc498686`. **27/27** offline checks and every `test_round.py` check pass.

	before	after
the round runner	2,504	**1,078**
executable statements	1,099	**556**
`frame_metrics`	279	**63**
`tissue_analysis.py`	611	**412**
a metric's definition	6 places, 4 files	**1 class**
LLM calls a round	153	**∼11**
python in `discovery_okuda`	23,725	**22,251**

Deleted: `round.py` (2,504), `agents/{llm_agents,proposer,metrologist}.py` (1,377), the post-hoc gates and two orphan checks (187), `predict.py`'s literal blocks (∼120), dead test cases (∼130). **Renamed:** `engine.py` → `round.py` (*engine* belongs to plexus2), `tube_analysis.py` → `tissue_analysis.py`.

The metrics: 67 defined, 24 in the bank, 5 headline

Cedric, 5 August: *"67 quantities is very large, in a previous effort we agreed on 24 . . . hold the classes but use only 24 in the loop and point the main important (5) to the agent so that*

it does not go into a given metric / rabbit hole."
The registry *defines* 67 because `euler`, `broken_n`, `ray_single_frac` and `hollow_frac` were the keys most likely to be computed where nobody could find them. It *hands over* 24. And it names five, one per question, so a role reads the whole round before any single number acquires an opinion:

`protr_peak`	is there a protrusion at all
`protrusion_aspect_max_peak`	a finger or a bulge — what no radius ratio can tell
`n_tubes_peak`	did the instrument call it a tube (zero, campaign-wide)
`act_cv_peak`	is there a pattern at all
`corr_act_rad_peak`	does the pattern grip the shape — *the* question

`metrics.Frame` caches the geometry that used to be built in the middle of the expressions using it — which is the whole reason `frame_metrics` was 279 lines — and the refactor is verified **bit-identical on three meshes, 66 keys**, including the `a_sw=None` fallback. The eight run-level scalars (`buf_full`, `mech_p_ratio`, `Q_drop`…) left the bank: they are evidence *context*, and `buf_full` turning true is a result about the array, not about biology.

Four defects the work exposed, none of them cosmetic

1. **No `edit_kind` passed to `critic.admit`.** `comp_hash` is parameter-blind by design, so once a parent is recorded *every retune of it* would be refused as a duplicate — a sweep impossible, most of a batch dead. Caught by the offline suite's own "a sweep is a legal experiment" case.
2. **And then the control.** Fixing that immediately refused slot 0, which *is* the parent unchanged — every round would have lost the one slot that makes the others interpretable, while reporting a full batch.
3. **A `set_param` on a bare operator name writes a key no operator reads.** `apply` does `g.params[k] = v` with no validation, so `rd_interface_tension.K_purse` (no node index) runs as an exact copy of the parent and is recorded as an experiment. The live Proposer wrote that form on its first real call and seven runs were submitted. Resolved by name where unique, and any edit that changes nothing is refused.
4. **The redacted menu.** `legal_menu` returns dicts; `[list(e) for e in rows]` yields their KEYS, so all 57 rows serialised as `["edit","label","yields","hash"]`. The Proposer said so in its reply — *"the menu came through fully redacted"* — and proposed blind.

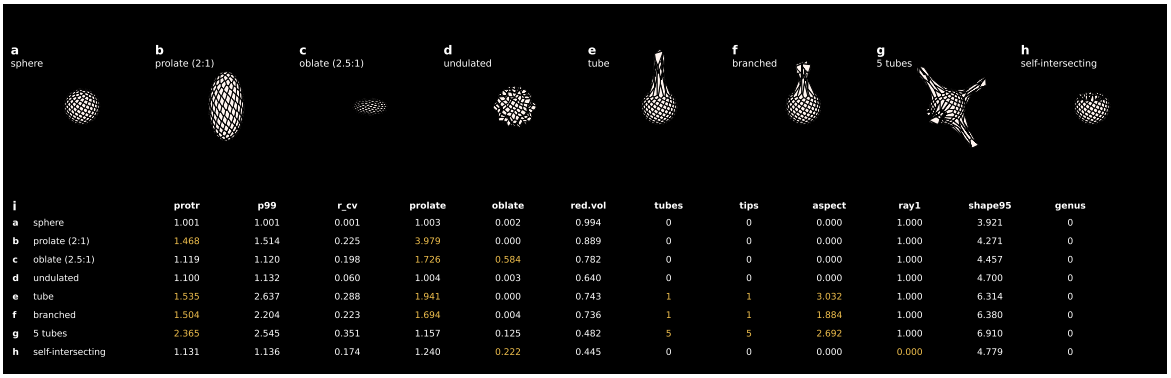
Numbers three and four were found by *reading what the agent said*, not what it returned, and both had already reached the cluster.

And one claim of mine that was false

The Analyst, given an empty history in a dry run, diagnosed round 2 as "growth against a frozen shell radius" — confidently, with numbers. I folded it into `round.md` as fact. It is wrong: `morphogen_growth_3d` has rescaled `R0` from the target volume since 31 July, five days before round 2 ran, and `r002c_03` broke P7 and P11 **with no growth operator at all**. A run that never grows cannot buckle from a growth radius. The real cause was the one already proven — `l_th_frac` 1.96 in eleven of twelve runs. Both the correction and the false diagnosis are now in `round.md`, the second so it is not proposed again.

Testing the ruler: metrics against generated geometries

Cedric, 5 August: *"can we test the metrics thoroughly against test geometries to be generated with a dedicated generator?"*, then *"test all metrics against known geometry"*, then — looking at the first figure — *"tube and branched gt are wrong, fix them."*
Every other test in this campaign checks that a metric *exists*, is admitted, has a producer and reaches a summary. None checked that it is **right**. That gap is not hypothetical: four metrics were measured to lie outright (F15/F16), `corr_act_rad` returned a confident 0.294 on a field whose entire spread was 8.4×10^{-5}, and the classifier called `coral_gate` *branched* where the eye saw a lobed sphere with `n_tubes` 0.
The generator deforms rather than builds. A metric needs valid topology — a closed trivalent sheet, every edge shared by exactly two faces — and that is already solved for a sphere, so all seven fixtures are `build_sphere_mesh` with their *vertices* moved. Connectivity is untouched, so `euler` and `genus` must stay exactly what a sphere gives and any change they report is their own bug rather than the fixture's.



	protr	p99	r_cv	prolate	oblate	red.vol	tubes	tips	aspect	ray1	shape95	genus
a sphere	1.001	1.001	0.001	1.003	0.002	0.994	0	0	0.000	1.000	3.921	0
b prolate (2:1)	1.468	1.514	0.225	3.979	0.000	0.889	0	0	0.000	1.000	4.271	0
c oblate (2.5:1)	1.119	1.120	0.198	1.726	0.584	0.782	0	0	0.000	1.000	4.457	0
d undulated	1.100	1.132	0.060	1.004	0.003	0.640	0	0	0.000	1.000	4.700	0
e tube	1.535	2.637	0.288	1.941	0.000	0.743	1	1	3.032	1.000	6.314	0
f branched	1.504	2.204	0.223	1.694	0.004	0.736	1	1	1.884	1.000	6.380	0
g 5 tubes	2.365	2.545	0.351	1.157	0.125	0.482	5	5	2.692	1.000	6.910	0
h self-intersecting	1.131	1.136	0.174	1.240	0.222	0.445	0	0	0.000	0.000	4.779	0

The eight fixtures and what the metrics say about each. Gold marks a metric firing on the shape it is meant to fire on. Panels are at a common scale — per-panel limits would make a tube and a sphere look equally elongated, hiding the difference the figure exists to show. Rendered with ***`run_tyssue_vesicle._draw`****, the same renderer as every movie and strip the campaign produces.*

What it found, including in my own work

1. `n_tubes` **= 3 on a rugby ball.** A 2:1 ellipsoid with no protrusion at all. It selects cells with $r > 1.3\times$ median and clusters them by direction, so on any elongated body it counts *the ends*, and the greedy angular clustering splits a broad polar cap into several. This is one of the **five headline metrics** — "did the instrument call it a tube" — answerable with 3 where there is no tube. Fixed by the definition rather than a constant: a tube is longer than it is wide, so a cluster with length $\leq$ diameter is not one. After: sphere 0, prolate 0, oblate 0, undulated 0, self-intersecting 0, tube 1, and exact counts to twelve on a many-tube fixture.

2. `n_tips` **has a 25-degree resolution, now measured.** Swept on the fork fixture, it flips $1 \to 2$ at a half-angle of 27° — exactly where the cone predicts. A narrower fork is *one* tip as far as the instrument is concerned, which matters because that is what early branching looks like. Pinned in both directions: the wide fork must read 2, the 12° fork must read 1, so if the cone ever changes, past "no branching" results are flagged as measured with a different ruler.

3. **Two fixtures of mine were wrong, and Cedric saw it in the figure before any metric complained.** The "tube" was a *teardrop* — a smoothstep taper from equator to

pole, which has neither the neck nor the constant width that `protrusion_aspect_max` and `tube_diameter` exist to measure. The "branch" was a *spindle*: it put a finger at each pole, which is two tubes on opposite sides of a body rather than a fork, and it applied its deformation twice to the same array in place, so the second pass moved vertices the first had already moved. The tube is now a cylinder mapped onto the polar cap; the branch is one trunk that splits, with the saddle pulled down into a real crotch — because a Y is not two tubes, it is two tubes *and* the junction between them.

4. `gyr_prolate` **rises on an oblate disc too**, to 1.73, because λ_3 collapses while λ_1 and λ_2 stay large. Correct arithmetic and a real trap: an agent told "`gyr_prolate` grows with elongation" would read a collapsing vesicle as a tube. `gyr_oblate` is what separates them (0.58 against 0.0002), so that is what the test asserts and the caveat is now in the metric's own docstring.

5. `ray_single_frac` **measures star-convexity, not self-intersection.** A wide fork is legitimately not star-convex — a ray grazing between the prongs can cross three times — so the branched fixture reads 0.990 with no fold anywhere in it. I expected this to mean P11 refuses the campaign's own target morphology. It does not: the threshold is 0.95, chosen for exactly this, and a folded shell reads 0.000. The margin is now a test, because if it ever tightens toward 1.0 branching becomes unreachable and nothing would say so.

6. `n_spots` **counts six as six** — but only once the fixture used maximally separated seeds. My first version drew six at random, got 3, and a loose bound accepted it: two random spots near each other legitimately merge, so that fixture could not distinguish an undercount from a merge.

Coverage: every metric, or a stated reason

The registry declares 67 quantities and the assertions touch about half, so the untested half was invisible — the shape of every defect this phase found. A metric is now either checked on a fixture or listed as `EXEMPT` *with a reason*, and the list is asserted exhaustive: adding a metric without doing one or the other fails the suite.

produced on a geometry	**56** / 67	asserted or present on at least one fixture
exempt, with a reason	13	8 run-level (need a series, not a frame), 4 rejected, 1 prose
`test_geometries.py`	**57 checks**	all passing

Second pass: did the phase keep its own discipline?

Cedric, 5 August: *"check that we are not over-complexifying vs plan in phase 12 … do not fall into the gap 'strong judge, weak learning' again."* Measured rather than asserted.
The trap was avoided. The refusal codes reachable from a launch are exactly the ten the phase planned — `R1`–`R4`, `R6`, `C1`–`C3`, every one structural: can this be built, is it new, can it reach its own target. Not one gate was added that judges a *result*. The eight remaining refusal paths in `round.py` are build failures (the parent will not rebuild, the edit is not applicable, the edit changed nothing, the spec will not write). Everything added since is an INPUT: `coverage`, `diagnosis`, the menu's `from`/`try`/`range`, and `range_notes` as text on the record rather than a veto.
But the runner doubled, and this note had stopped saying so. 343 → 556 statements, 660 → 1,078 lines. The figures above said 480 until this pass corrected them — a note that

has stopped describing the code is the defect this campaign keeps paying for, and it does not become acceptable for being in a note I wrote.

+1,319 lines since the phase commit, and where they went		
`test_geometries.py`	591	the verification the phase said was missing
`round.py`	503	four requested features, three measured defects
`figure_geometries.py`	124	the figure
`test_round.py`	113	regressions for the defects
everything else	100	ports, budgets, voices
tests and figure	**828**	63% of the growth is verification, not loop

Traced individually, the runner's growth is the CLI and campaign reset (78 statements), the coverage node (24), the menu grid (48), the spec-fidelity overlay (36), the no-op and name resolution (19) and the pool pre-flight (20). Each answers a request or a measured failure; none is speculative. One helper existed for seven statements used once and has been folded back into its caller.

What is still unbudgeted. `metrics.py` is 1,141 lines for 67 classes, against the ∼120 lines of literal lists it replaced — it removed the six-place edit and gave the metrics a tested definition, but it has not paid for itself in line count and should not be described as if it had. And 32% of `round.py` is prose. That is a deliberate trade, not an accident, but it is the reason the file reads as twice the size the statement count implies.

What you get. A loop of four roles and ∼11 calls a round that keeps the three things the control cannot have — composition, a structural critic, and an eye — and drops the eleven that were judging without learning.

Phase 13 — The operator battery: is this parameter connected? NOT STARTED

Goal. No parameter enters a campaign unmeasured. For every operator and every parameter, establish whether changing it changes the run — and use the same harness to turn 87 declared differentiability flags into 87 measurements.

Why: three layers of defence, and all three missed the same thing

`shape_to_chem` is the operator that closes Okuda's loop — geometry back to chemistry. It was edited **25 times across 13 rounds**, 8 of them same-seed. Not one of those 8 changed the run by a single bit. Every one was recorded as `refuted`, which is the campaign asserting a mechanism claim about an operator that never reached the state.

Three things should have caught it:

1. **The operator's own self-test, which passes.** Forty lines certifying that curvature reads $1/R$ on spheres of known radius, that a bump is positive and a dimple negative, that the standardisation is unit-invariant, that one spike cannot set the scale. All correct, and all about arithmetic *inside* `_feature()`. Its one check about `forward` is literally `chk(True, "beta = 0 returns zeros by construction")` — an assertion hardcoded to pass.

2. `instrument.install()`, whose entire purpose is "did this operator actually do anything?". It decides with `acted = bool(out) or (before != after)`. `shape_to_chem` returns a full-size tensor on every call, so `bool(out)` is `True` *even when that tensor is all zeros.* It has scored 100% acted on every run it has ever been in.

3. **The scoring itself**, which cannot distinguish *the hypothesis is false* from *the knob is disconnected*. At least 13 of the campaign's 84 refutations refuted nothing.

The common failure is exact: **none of the three measured the operator's effect on the trajectory.** One measured its internal arithmetic, one measured whether it returned an object, one measured a metric downstream of it.

Three probes, cheapest first

`prototype/Tyssue/op_probe.py`:

probe	cost	catches	verdict
`UNREAD`	free, no simulation	the class never looks the key up	dead by construction
`TRAJECTORY`	2 short runs	read, but cannot reach the state	`LIVE` / `DEAD`
`GRADIENT`	1 run with the tape	*how much*, and where the path breaks	sensitivity

`UNREAD` hands the operator a `params` dict that records every key read during `__init__` and `forward`; a declared key never read cannot influence anything, whatever the sweep does to it. Run against the r013 control it took under a second and returned `cell_rd_seed.amp`, `morphogen_growth_3d.dt` and `seed_mesh_3d.cell_set` — three parameters that have sat in every spec this campaign has written and are read by nothing.
`TRAJECTORY` is deliberately dumber than all three failed defences: run the composition twice, change one number, compare the state. It cannot be fooled by an emission that is discarded downstream, because it never looks at the emission. It compares *state*, not metrics — a metric can be blind to a change (`n_spots` reads 1 on a six-domain pattern) and then report a live parameter dead.

Does this need autograd? Not for the fix — but it is exactly the groundwork

The finite difference is the ground truth and must stay: it cannot be fooled by a severed tape, and it works on operators that have no gradient at all. Autograd adds two things it cannot give.
First, ranking instead of classification. `TRAJECTORY` answers *live or dead*; $\partial(\text{state})/\partial\theta$ answers *how much, and in which direction*, from one run instead of two per parameter. That is what turns the battery from a defect detector into a menu ordering — the Proposer's real problem is not which knobs are dead but which of the live ones is worth a GPU.
Second, and this is the differentiability work itself: the interesting cell of the table is *finite difference nonzero, gradient zero*. That is an operator that is live but not *learnable*, and the probe names the barrier. The barriers here are known and specific: `_standardise` computes the shape feature in **numpy** (`np.median`, `np.clip`), which severs the tape outright; `shape_to_chem` then applies `clamp(min=0, max=F_CEIL)`, which zeroes the gradient wherever it binds — and at $\beta = -2$ with the feature clipped to ± 4 it binds on most cells; `cell_rd_seed` uses `argmax` and `torch.where` on a hard radius.
87 implementations are registered; 23 declare `DIFFERENTIABLE = False` — including *all four* `shape_to_chem` implementations and `morphogen_growth_3d`, which is to say both operators of the chemistry–shape loop. Those flags are *declarations*. Nobody has measured one. The battery makes each of them a measurement, and the three-way outcome (gradient matches the finite difference / gradient is zero where the difference is not / no tape at all) is a work list for making them differentiable, ordered by how much each one moves the run.

The fixes this exposes, none of them cosmetic

1. `cell_rd_seed` carries no `before_frame`, so the *initial condition* is re-applied every frame, overwriting both chemistry channels. `shape_to_chem` writes to channel 1, which the reset pins to exactly 1.0. **Measured, 140 frames:** as the campaign ran it, $\beta : 0 \to -2$ and $0 \to -4$ both move the trajectory by **exactly** 0; with the seed gated to `before_frame: 1` they move it by 2.34×10^{-5} and 2.38×10^{-5}.

2. **And a second throttle the same measurement exposes.** Doubling β changes the effect by 1.8%. A coupling should scale with its strength; this one does not, because `clamp(min=0, max=F_CEIL)` saturates — with the feature clipped to ± 4, $F_0(1 + \beta\hat{\phi})$ leaves $[0, 0.11]$ on most cells at both -2 and -4, so the two clamp to the same field. Gating the seed removes one throttle of two. *This is also the clearest case for the gradient probe: a saturated clamp has zero derivative, so* ***`shape_to_chem`*** *is the canonical "live but not learnable" entry in the table above.*

3. `instrument.py`: `bool(out)` must become "the emission contains a nonzero value".

4. `cell_rd_seed.n_spots` is read only by `_cone_dirs()`, which only `mode: cones` calls; the campaign runs `mode: tip`. A parameter that is inert in the current mode should be refused at build time, not swept.

5. `amp` is read by no branch of any mode.

What you get. A battery every operator runs from its own `__main__`, a `LIVE/DEAD/UNREAD` column on every parameter in the menu before a round can propose it, and the measured differentiability table that the inverse work needs.

Phase 14 — The demonstration NOT STARTED

Goal. Run the designed grid — six to eight simulations — that produces the figure: tube, undulation and branching, at Okuda's settings.
What you get. *The figure.*

Phase 15 — The method claim NOT STARTED

Goal. Write up what the loop found by itself versus what we told it, with the record of predictions and retractions as the evidence.
What you get. The argument that Track A works, resting on Track B.

Phase 16 — The audit: 28 rounds, and the campaign had no organ DONE

Goal. Twenty-eight rounds ran. Ask an outside auditor why they did not add up, and rebuild the loop on the answer. *Out of order deliberately: this ran before 13–15, because a battery of measurements is worth little to a loop that cannot act on them.*

The complaint, and the verdict

Cedric, 10 August: running 28 rounds is a success, but the loop "does not get the global picture, goes in rabbit holes, duplicates too much, and does not explore with imagination, serendipity or prior knowledge." An external audit of the code and the 416-record ledger returned a one-line verdict: this is **a well-instrumented parameter-refinement rig wearing the interface of a discovery engine**. What makes a *round* trustworthy — the premise gate, the deterministic Critic, the pre-registered prediction, the eye as an independent wit-

ness — is good and was left alone. What is missing is everything *above* the round. There is no organ that holds the campaign.

Four things nobody had measured

1. **The memory was a tail.** `history` is `knowledge.md` read with `limit=12000`, and `_read` keeps the *last* 12,000 characters of a 38,591 character file. **Rounds 1–20 were invisible to every role, in every round**, with no truncation marker, because the cut happened before the block that prints one.

2. **Three declared edges carried nothing.** `load_flow` verifies that every emitted key appears in some node's `in:`; it cannot verify the node *uses* it. The Proposer declared `refusals` and `user_input`, the Analyst declared `user_input` and `route_a_results`, and no crew module mentioned any of them. So `campaign/user_input.md` — the human's only channel into the loop — **reached no role for 28 rounds**, and 220 Route A slots produced response curves handed to nobody.

3. **The rabbit hole was a sort key.** Parents were ranked `-grip_peak, -protr_peak`: one scalar, no diversity, novelty, age or lineage term. Measured: 18 distinct compositions across 196 structural records, **one composition proposed 87 times**, `r020_06` the parent of 33 slots, and for four consecutive rounds the top six parents were *five clones of one result* at `protr_peak` 1.595.

4. **A hard-coded constant hid the campaign's best result.** `FORCED_P_RATIO = 2.0` sorted any run with `mech_p_ratio` above it dead last, permanently. Eleven runs crossed it and **not one was ever used as a parent** — among them `r019_07`, `protr_peak` **1.817** and `grip_peak` **0.294**, the highest of both in the whole campaign, against a control ceiling of 1.595 / 0.262 that four rounds could not beat. The loop's best result was structurally unreachable, and the proxy was standing in for a forcing term that measured **0.0 in all 78 specs that ever carried it**.

The purse-string and the push were one operator

`rd_interface_tension` carried $E = K_{\text{purse}} \sum_{\text{iface}} \ell_e - K_{\text{extrude}} \sum_{\text{red}} a\, r$. The first term is a line tension on the activator interface: ordinary physics. The second is an energy that *falls* as activated cells move outward — the answer written into the objective. One plausible name over both made forcing a *parameter of a sound operator*, and the cost was four rounds of "extrude-forced" verdicts on runs whose specs contain no such operator at all. They are two operators now: `interface_line_tension_3d`, which is in the vocabulary and has no `K_extrude` key to set, and `extrusion_forcing_3d`, which is not in `composition_space` at all. There is no longer a setting of anything in the search space that pushes.

Death was legal, reachable and never chosen

`apoptosis_3d` — the Die family, and the first mechanism here that deforms the sheet *inward* — was injected before round 25. For five rounds it sat on the Proposer's menu and was named by `coverage` as the only operator never exercised, and it was **never once chosen**. The reason is in the edit log: `add_op` has fired **30 times in the campaign's history and all 30 added the same operator**, none since round 24. Route A could not reach it either, because `_build_sweep` refuses a base that does not already carry the operator, and the Route A plan is static. **A mechanism enters this campaign through the basis or not at all**, so the basis now carries a death twin of each Route A base.

What was rebuilt

Every policy number moved out of the engine and into `crew/flow.yaml`, which is what the graph is for:

change	what it fixes
edges wired; `load_flow` now refuses a declared agent input the role does not read	the human channel and the Route A curves arrive; negative-tested against a planted fake edge
`grounding` is a node, feeding the Proposer	the Grounder was terminal, writing the round's most strategic sentence into a file nothing read
`history` limit `null`	the first twenty rounds become visible again
parents are a **portfolio**, one reserved seat per target morphology, plus `max_per_lineage`	the rabbit hole closes mechanically, without asking a role to be more imaginative
`forced_p_ratio: null`	forcing is settled structurally; the proxy could only produce false positives
basis → 16 members with death, at 1800 frames	ten members had been running at *half* the length of their own children

The portfolio and Cedric's new objective — "explore altogether tube forming, budding, branching, complex shape" — are the same fix. One scalar can only climb one hill. Each target now holds a seat and its own figure of merit: `tube` on aspect ratio, `bud` on protrusion with aspect counting *against* it (a bud is not a failed tube), `branched` on tip count, `complex` on grip and invagination. Replayed on the archived ledger, the old rule returns five clones; the new one returns a tube, a bud, a branched specimen and `r019_07`.

What is still missing, and named so it is not mistaken for done

The audit's remaining recommendations are campaign-level organs, each a code node and an edge rather than a new role:

- **No serendipity organ.** Nothing notices a result that was interesting but was not what the round was testing. `hypothesis.py` implements one — surprise rate, novelty yield, an intent-mix control band — in 34 kB that **nothing imports**. It was left on the floor by the Phase 12 reduction: the loop had a learning-rate governor and lost it in a refactor.
- **No global hypothesis.** Every proposal is (parent, edit, one metric, one threshold). A claim spanning runs — "grip is set by the diffusion *ratio*, not by d_a" — cannot be posed, let alone scored.
- **Duplication is free.** A refused duplicate is re-admitted at a fresh seed, relabelled `replicate`, and scored against the copied prediction. It was 5 of 7 Route B slots in r028.
- **Prior knowledge is one-way.** `_premises_raw.md` holds 70 literature-sourced facts; 11 became gates and 59 are unread — including *tissues stop growing when compressed*, a direct diagnosis of this project's 30,743-cell overshoot.

What you get. A campaign that starts from an empty ledger, a 16-member basis that includes cell death, a parent set that cannot collapse onto one lineage, and a search space in which no composition can be paid to produce the answer.

6. The 31 July lesson, in three lines

Ten defects in one day, every one found by Cedric looking at a picture and none by the loop — a ball that shrank to nothing, a movie where every cell turned green at once, a coral that began uniformly red. They were all the same kind: the loop could tell whether a *simulation* succeeded and had never been able to tell whether the *specimen* was a tissue. Phase 1 is the answer to exactly that, and the only way to find out whether it is enough is to run a batch and see whether the gate catches anything before Cedric does.

7. What I need from you

Nothing blocking. Two things worth knowing:

1. **The buffer gate is the one decision worth taking now.** Phase 5 has already been spent once on runs that could not have produced evidence. A pre-launch refusal costs nothing and would have saved the batch.
2. **Tell me if this note is the right shape.** Too long, too short, wrong order — it is cheap to change, and it is the thing that lets you steer.

Appendix — words I could not avoid

Only these four. Everything else above is ordinary English.

- **Cell sheet (or mesh)** — the model of the tissue: a hollow ball of polygonal cells sharing walls, like a football.
- **Operator** — one mechanism as a reusable building block: "cells divide", "chemical spreads", "surface pulls tight". A model is a combination of these.
- **The loop** — the automated cycle: propose a change, predict the outcome, run, measure, record whether the prediction held.
- **Phase** — one of the eight stages above. Our agreed stopping points.

The four measurements, renamed

Agreed 31 July. The old names misled in three separate ways and are being retired: `protr` reads as a length but is a *ratio*; `final` became ambiguous the moment we introduced the evidence horizon; and `Q` said nothing at all. The rename happens as one mechanical pass once the current work lands, with old records aliased on read — the archive is never rewritten.

was	is	means
`protr`	`elongation`	the 95th-percentile cell radius divided by the median. **1.0 is a sphere.** A ratio, not a length — reading 1.62 as an amount of protrusion is the mistake the old name invited.
`protr_peak`	`elongation_peak`	the most elongated it ever got, over *valid* frames only.
`protr_final`	`elongation_at_end`	at the last *valid* frame — not simply the last frame.
`Q`	`elongation_unforced`	what remains after growth and pushing are switched off and the tissue relaxes. **This is the forced-versus-grown test**, and it is the whole reason the campaign exists: we can already make a tube by pushing; the question is whether the tissue will hold one by itself.

My working log is `SESSION_LOG.md`; the findings register is `findings.py` and the premises are `PREMISES.md`. You should not need any of them.